

BrickPick — User Guide and Theoretical Manual

Autonomous brick-picking system for a RoboMaster EP mobile manipulator

Version 1.0 · May 2026

Table of Contents

- [Part I: User Guide](#)
 - [1. System Overview](#)
 - [2. Hardware Requirements](#)
 - [3. Software Installation and Build](#)
 - [4. Complete Operational Workflow](#)
 - [5. Configuration Reference](#)
 - [6. Running Individual Components](#)
 - [7. Troubleshooting](#)
- [Part II: Theoretical Manual](#)
 - [8. Architecture and Design Rationale](#)
 - [9. Map Partitioning Pipeline](#)
 - [10. Vision and Detection](#)
 - [11. Object Search with Multi-Frame Debounce](#)
 - [12. Approach Controller with Blind-Zone Takeover](#)
 - [13. Behavior Tree Orchestration](#)
 - [14. Inter-Process Communication Protocol](#)
 - [15. Arm Preset Sequencing](#)
- [Appendix A: Parameter Tuning Guide](#)
- [Appendix B: File Reference](#)

Part I: User Guide

1. System Overview

BrickPick enables a RoboMaster EP mobile robot to autonomously navigate an environment, detect brick-like objects using a YOLO vision pipeline, approach and grasp them with a robotic arm, and repeat this process across multiple partitioned regions of a map.

Core capabilities:

Capability	Implementation
Object detection	YOLO inference on RGB camera frames
Map partitioning	Offline Voronoi segmentation with A*-based merging
Region traversal	Nearest-neighbor TSP ordering
Navigation	Nav2 stack with SLAM localization
Object search	In-place rotation with multi-frame debounce

Capability	Implementation
Object approach	Yaw-alignment + forward drive + blind-zone takeover
Arm control	Preset-sequence execution via action client
Mission orchestration	BehaviorTree.CPP with custom ROS 2 nodes

2. Hardware Requirements

Component	Specification
Robot platform	RoboMaster EP (or S1)
Onboard computer	NVIDIA Jetson or x86 running Ubuntu 22.04
LiDAR	YDLIDAR (any model supported by ydlidar_ros2_driver)
Camera	RoboMaster EP built-in camera
Arm	RoboMaster EP mechanical gripper arm

3. Software Installation and Build

3.1 System Dependencies

```
# Ubuntu 22.04 with ROS 2 Humble (full desktop install recommended)
# https://docs.ros.org/en/humble/Installation.html

# Python dependencies
pip install ultralytics scikit-image scipy opencv-python pyyaml

# BehaviorTree.CPP v4.x
# https://github.com/BehaviorTree/BehaviorTree.CPP

# RoboMaster SDK (use Ultra fork if version conflicts arise)
# https://github.com/RamessesN/Robomaster-SDK-Ultra

# RoboMaster ROS bridge
# https://github.com/RamessesN/robomaster_ros

# YDLIDAR SDK and ROS 2 driver
# https://github.com/YDLIDAR/ydlidar_ros2_driver

# Nav2 and slam_toolbox
sudo apt install ros-humble-nav2-bringup ros-humble-slam-toolbox
```

3.2 Building the Package

```
mkdir -p ~/brickpick_ws/src
cd ~/brickpick_ws/src
# Clone or copy the BrickPick package into src/

cd ~/brickpick_ws
```

```
colcon build --packages-select brickpick --symlink-install
source install/setup.bash
```

The `--symlink-install` flag allows editing Python scripts without re-building. Add the `source` command to your `~/.bashrc` for convenience.

3.3 Prepare YOLO Model Weights

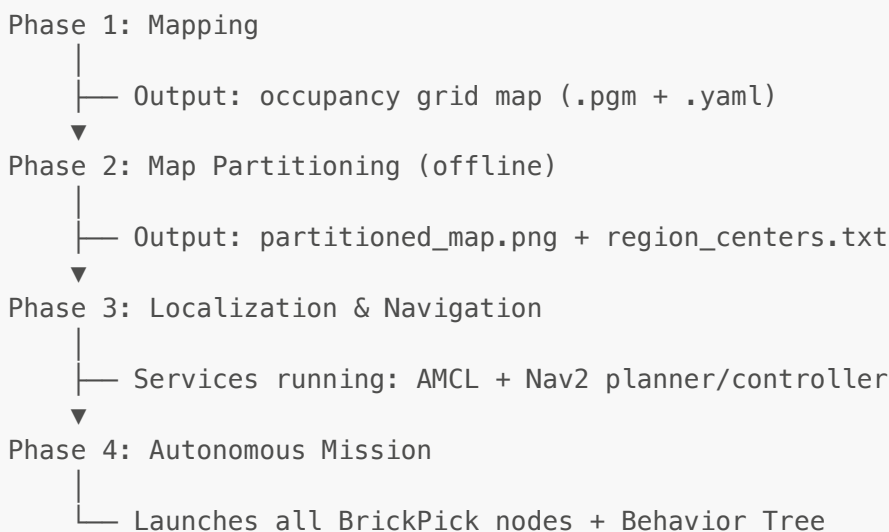
Before launching any vision-related node, place your trained YOLO model weights (`.pt` file) in the `model/` directory. The default path is configured in `config/vision_params.yaml` (`model_path: "best.pt"`). The package ships with two model variants:

- `model/best.pt` — current active model
- `model/best(old).pt` — previous version kept as fallback

4. Complete Operational Workflow

The end-to-end workflow consists of **four sequential phases**, each depending on the outputs of the previous phase. A fifth section covers debugging and one-off tasks.

Phase dependency graph:



Phase 1: Mapping (SLAM)

Objective: Build an occupancy grid map of the environment.

Prerequisites: YDLIDAR connected and powered on; RoboMaster EP powered on and connected via Wi-Fi (STA mode).

Step 1.1 — Start the 2D odometry filter.

This node must run **before** SLAM because it publishes the `odom_2d` → `base_link` transform that `slam_toolbox` consumes. Without it, the SLAM node will have no odometry input:

```
ros2 run brickpick odom_2d_filter.py
```

Step 1.2 — Start the RoboMaster robot bridge and LiDAR driver.

Open separate terminals for each:

```
# Terminal A: Robot bridge (connects to RoboMaster EP via Wi-Fi)
ros2 launch robomaster_ros main.launch model:=ep conn_type:=sta

# Terminal B: YDLIDAR driver (if not already included in the robot launch)
# Refer to your specific YDLIDAR model's launch instructions
```

Step 1.3 — Launch SLAM and teleoperation.

```
# Terminal C: Online async SLAM
ros2 launch slam_toolbox online_async_launch.py \
  use_sim_time:=false \

slam_params_file:=/home/nvidia/brickpick_ws/src/brickpick/config/mapper_params_on
line_async.yaml

# Terminal D: Keyboard teleoperation for exploration
ros2 run brickpick teleop_keyboard_node.py
```

Step 1.4 — Explore and save the map.

Drive the robot through the entire environment using the keyboard teleoperation node (see Section 6.4 for controls). Cover all areas the robot will later need to navigate. Once satisfied:

```
# Save the map (use slam_toolbox's map saver or the standard map_server CLI)
ros2 run nav2_map_server map_saver_cli -f ~/map/livingroom_clean_12
```

This produces two files:

- `livingroom_clean_12.pgm` — the occupancy grid image
- `livingroom_clean_12.yaml` — metadata (resolution, origin, thresholds)

Note: The `map_partitioner.py` script is pre-configured to read `map/livingroom_clean_12.yaml`. If you name your map differently, update the `yaml_file` variable in the `__main__` block of `map_partitioner.py`.

Phase 2: Map Partitioning (Offline)

Objective: Divide the map into reachable regions and compute the optimal visitation order.

Prerequisites: Phase 1 completed (map `.pgm` and `.yaml` files exist).

Step 2.1 — Verify the home pixel coordinate.

Open `map_partitioner.py` and confirm the `home_py`, `home_px` values in the `__main__` block correspond to the robot's starting position on the map. These are hardcoded pixel coordinates:

```
home_py, home_px = 288, 48 # Adjust to your map
```

Step 2.2 — Run the partitioner.

```
python3 brickpick/map_partitioner.py
```

Step 2.3 — Review the outputs.

Three artifacts are generated in the working directory:

Output File	Description
occ_map_visualization.png	Diagnostic: raw occupancy grid with home position marked
partitioned_map.png	Color-coded regions with numbered centers (red crosses) and TSP order
region_centers.txt	World-coordinate list of region centers in TSP visitation order

The `region_centers.txt` format:

```
# Region Centers (World Coordinates: x y)
1.234 5.678
2.345 6.789
...
```

The Behavior Tree's `ReadRegionCenters` node parses this file at runtime.

Step 2.4 — Tune partitioning parameters (if needed).

If regions are too large, too small, or the TSP order seems suboptimal, adjust the parameters in the `MapPartitioner(...)` constructor call inside the `__main__` block (see Section 5.5) and re-run.

Phase 3: Localization & Navigation

Objective: Start AMCL localization and the Nav2 navigation stack so the robot can autonomously navigate to region centers.

Prerequisites: Phase 2 completed (`region_centers.txt` exists); map files in place.

Step 3.1 — Start the robot bridge and odometry filter.

Same as Phase 1, Step 1.1–1.2:

```
# Terminal 1: Robot bridge
ros2 launch robomaster_ros main.launch model:=ep conn_type:=sta

# Terminal 2: 2D odometry filter (must run before localization)
ros2 run brickpick odom_2d_filter.py
```

Step 3.2 — Start localization (AMCL).

```
# Terminal 3: AMCL localization
ros2 launch nav2_bringup localization_launch.py \
  map:=/home/nvidia/map/livingroom_clean_12.yaml \
  params_file:=/home/nvidia/brickpick_ws/src/brickpick/config/nav2_params.yaml \
  use_sim_time:=false
```

Wait until the robot is localized. Verify with:

```
ros2 topic echo /amcl_pose
```

Step 3.3 — Start navigation.

```
# Terminal 4: Nav2 navigation stack
ros2 launch nav2_bringup navigation_launch.py \
  params_file:=/home/nvidia/brickpick_ws/src/brickpick/config/nav2_params.yaml \
  use_sim_time:=false
```

Important: Both `localization_launch.py` and `navigation_launch.py` must share the same `params_file` so their costmaps and planner configurations are consistent.

Phase 4: Autonomous Mission Execution

Objective: Run the full BrickPick pipeline — the Behavior Tree engine orchestrates navigation, object finding, approach, and arm grasping across all partitioned regions.

Prerequisites: Phase 3 running (localization + navigation services are healthy).

Step 4.1 — Launch the full pipeline.

```
ros2 launch brickpick main.launch.py
```

This single launch file starts all five core nodes:

Node	Executable	Function
brickpick_vision	vision_node.py	YOLO inference on camera frames
find_node	find_node.py	Rotation search with multi-frame debounce
approach_node	approach_node.py	5-state approach controller
arm_preset_node	arm_preset_node.py	Preset-sequence arm control
brickpick_bt_executor	bt_executor	Behavior Tree engine (50 Hz tick)

Step 4.2 — Monitor execution.

The BT executor logs each node transition to the console. You can also monitor individual node statuses:

```
# Watch detection output
ros2 topic echo /vision/detections

# Watch find/approach/arm status
ros2 topic echo /find_node/status
ros2 topic echo /approach_node/status
ros2 topic echo /arm_preset_node/status
```

Step 4.3 — Using a custom Behavior Tree.

Two tree variants are included for different use cases:

```
# Full multi-region traversal with local pickup loops (default)
ros2 launch brickpick main.launch.py

# Single find-approach-grab cycle for debugging
ros2 launch brickpick main.launch.py bt_xml_path:=config/simple_pick_tree.xml

# Custom tree
ros2 launch brickpick main.launch.py bt_xml_path:=/path/to/custom_tree.xml
```

Tree	File	Purpose
Full mission	config/brickpick_tree.xml	Multi-region traversal, global + local pickup loops
Simple debug	config/simple_pick_tree.xml	Single Find→Approach→Arm cycle, no navigation
Arm-only debug	config/simple_pick_tree.xml (DebugArmTree)	ExecuteArmSequence only

Step 4.4 — Expected behavior.

The robot will:

1. Read region_centers.txt and begin at index 0
2. Navigate to each region center via Nav2
3. At each center: rotate to find an object → align and approach → extend blindly → execute the arm grab sequence
4. After the global grab: enter a local re-search loop to pick up any remaining visible objects
5. Move to the next region and repeat
6. After all regions are exhausted: navigate back to the starting position

The mission continues until all regions are processed or a fatal navigation failure occurs.

5. Configuration Reference

5.1 Vision Parameters (config/vision_params.yaml)

Parameter	Default	Description
<code>model_path</code>	<code>best.pt</code>	Path to YOLO weights (relative to package share)
<code>device</code>	<code>cuda:0</code>	Inference device: <code>cpu</code> , <code>cuda:0</code> , etc.
<code>camera_topic</code>	<code>camera/image_color</code>	ROS topic for camera frames
<code>conf_threshold</code>	<code>0.5</code>	Minimum confidence for a detection to be published
<code>iou_threshold</code>	<code>0.45</code>	NMS IoU threshold
<code>imgsz</code>	<code>640</code>	YOLO inference image size
<code>publish_debug_image</code>	<code>true</code>	Whether to publish annotated debug frames

5.2 Approach Parameters (`config/approach_params.yaml`)

Parameter	Default	Description
<code>img_width / img_height</code>	<code>640 / 360</code>	Camera resolution (must match actual)
<code>target_x_offset</code>	<code>370.0</code>	Desired horizontal pixel position of target (center of gripper)
<code>yaw_kp</code>	<code>0.002</code>	Proportional gain for yaw correction
<code>forward_speed</code>	<code>0.1</code>	Linear speed during approach (m/s)
<code>stop_y_threshold</code>	<code>280.0</code>	Pixel y-coordinate at which the target enters the blind zone
<code>align_threshold</code>	<code>15.0</code>	Pixel tolerance for yaw alignment
<code>timeout_lost</code>	<code>1.0</code>	Seconds without detection before reporting failure
<code>extend_dist</code>	<code>0.30</code>	Blind extension distance after target enters blind zone (m)

5.3 Arm Presets (`config/arm_presets.yaml`)

Parameter	Description
<code>presets.home.x / presets.home.z</code>	Home (retracted) position
<code>presets.forward.x / presets.forward.z</code>	Forward (extended) position
<code>presets.down.x / presets.down.z</code>	Downward position for grasping
<code>presets.backward.x / presets.backward.z</code>	Retraction after grasp
<code>default_sequence</code>	Ordered list of preset names to execute
<code>position_limits.*</code>	Joint travel limits for validation
<code>emergency_stop_on_error</code>	Return to home on sequence failure

5.4 Find Parameters

Set via launch arguments or parameter overrides:

Parameter	Default	Description
<code>rotate_speed</code>	<code>0.6</code>	Angular velocity during search (rad/s)

Parameter	Default	Description
<code>required_confirm_frames</code>	4	Consecutive detection frames required to confirm
<code>min_confidence</code>	0.6	Higher threshold for search confirmation vs. raw detection

5.5 Map Partitioner Parameters

Set in the `__main__` block of `map_partitioner.py`:

Parameter	Default	Description
<code>inflation_radius</code>	3	Obstacle inflation in pixels (robot radius)
<code>min_area</code>	300	Minimum region area in pixels before forced merge
<code>alpha</code>	1.0	Area penalty weight in merge cost function
<code>beta</code>	0.002	Distance penalty weight in merge cost function
<code>merge_cost_threshold</code>	2.0	Maximum cost for a merge to be accepted
<code>max_region_radius</code>	1	Hard radius constraint (meters)

6. Running Individual Components

Each node can be run standalone for testing, debugging, or data collection without launching the full pipeline.

6.1 Vision Node

Useful for verifying model accuracy and tuning confidence thresholds:

```
# Run on CPU
ros2 run brickpick vision_node.py --ros-args -p device:="cpu"

# Run on GPU with custom model
ros2 run brickpick vision_node.py --ros-args -p device:="cuda:0" -p
model_path:="/path/to/model.pt"

# View annotated output in rqt or RViz
ros2 run rqt_image_view rqt_image_view /vision/annotated_image
```

6.2 Find Node (Standalone)

Manually trigger the find behavior for testing:

```
ros2 run brickpick find_node.py --ros-args -p rotate_speed:=0.4
# In another terminal, trigger the behavior:
ros2 service call /find_node/start std_srvs/srv/Trigger
```

6.3 Approach Node (Standalone)

Test the approach controller in isolation:

```
ros2 run brickpick approach_node.py
# In another terminal:
ros2 service call /approach_node/start std_srvs/srv/Trigger
```

6.4 Arm Preset Node (Standalone)

Verify arm preset positions without the full tree:

```
ros2 run brickpick arm_preset_node.py
# Trigger the sequence:
ros2 service call /arm_preset_node/start std_srvs/srv/Trigger

# Use the DebugArmTree for arm-only testing:
ros2 launch brickpick main.launch.py bt_xml_path:=config/simple_pick_tree.xml
```

6.5 Dataset Image Capture

Collect training images for YOLO model fine-tuning. Press **s** to save the current frame:

```
ros2 run brickpick image_capture_node.py --ros-args -p save_dir:="/dataset"
```

Captured frames are saved as timestamped PNG files in the specified directory. A **dataset.zip** is included in the repository with pre-collected samples.

6.6 Keyboard Teleoperation Controls

The **teleop_keyboard_node.py** uses a TurtleBot3-style incremental control scheme:

Key	Action
w / x	Increase / decrease forward speed
a / d	Increase / decrease angular speed
s or Space	Stop (force zero)

7. Troubleshooting

Robot does not move during "Find" phase

1. Verify **vision_node** is publishing detections: **ros2 topic echo /vision/detections**
2. Check **find_node** parameters — **min_confidence** may be too high
3. Verify the camera topic name matches between **vision_params.yaml** and the actual robot

Approach fails before reaching the target

1. Check **timeout_lost** — if the target is temporarily occluded, increase this value
2. Verify **stop_y_threshold** — if the target disappears too early (before entering the blind zone), increase it
3. Check that **target_x_offset** corresponds to the gripper's horizontal position in the image

Navigation fails to reach region centers

1. Verify the map is correctly localized: check `/amcl_pose` and compare with `region_centers.txt`
2. Ensure `nav2_params.yaml` planner and controller are configured for your robot's dynamics
3. Check that region centers are in free space and reachable via the A* paths computed during partitioning

Arm sequence fails

1. Verify the RoboMaster action server is available: `ros2 action list | grep move_arm`
2. Check that preset positions are within the configured `position_limits`
3. Try the simple arm debug tree: `ros2 launch brickpick main.launch.py`
`bt_xml_path:=config/simple_pick_tree.xml`

Build errors

1. Ensure BehaviorTree.CPP v4.x is installed and findable by CMake
2. Verify all ROS 2 package dependencies are installed: `rosdep install --from-paths src --ignore-src -r -y`
3. For Python import errors after build, verify `source install/setup.bash` was run

Part II: Theoretical Manual

8. Architecture and Design Rationale

BrickPick employs a **hybrid architecture** combining deliberative planning (Behavior Tree mission orchestration) with reactive control (PID-based approach, state-machine-driven search). The system follows three key design principles:

8.1 Separation of Concerns via Service-Based IPC

All "smart" behaviors (find, approach, arm sequence) are implemented as standalone Python ROS 2 nodes. Each exposes a standard `std_srvs/Trigger` service (`~/start`) and publishes status on a `std_msgs/String` topic (`~/status`). The C++ Behavior Tree engine calls these services and polls status topics.

Rationale: This decoupling allows each behavior to be developed, tested, and debugged independently. The BT engine never needs to understand the internals of a perception or motion task — it only needs to know whether it succeeded or failed.

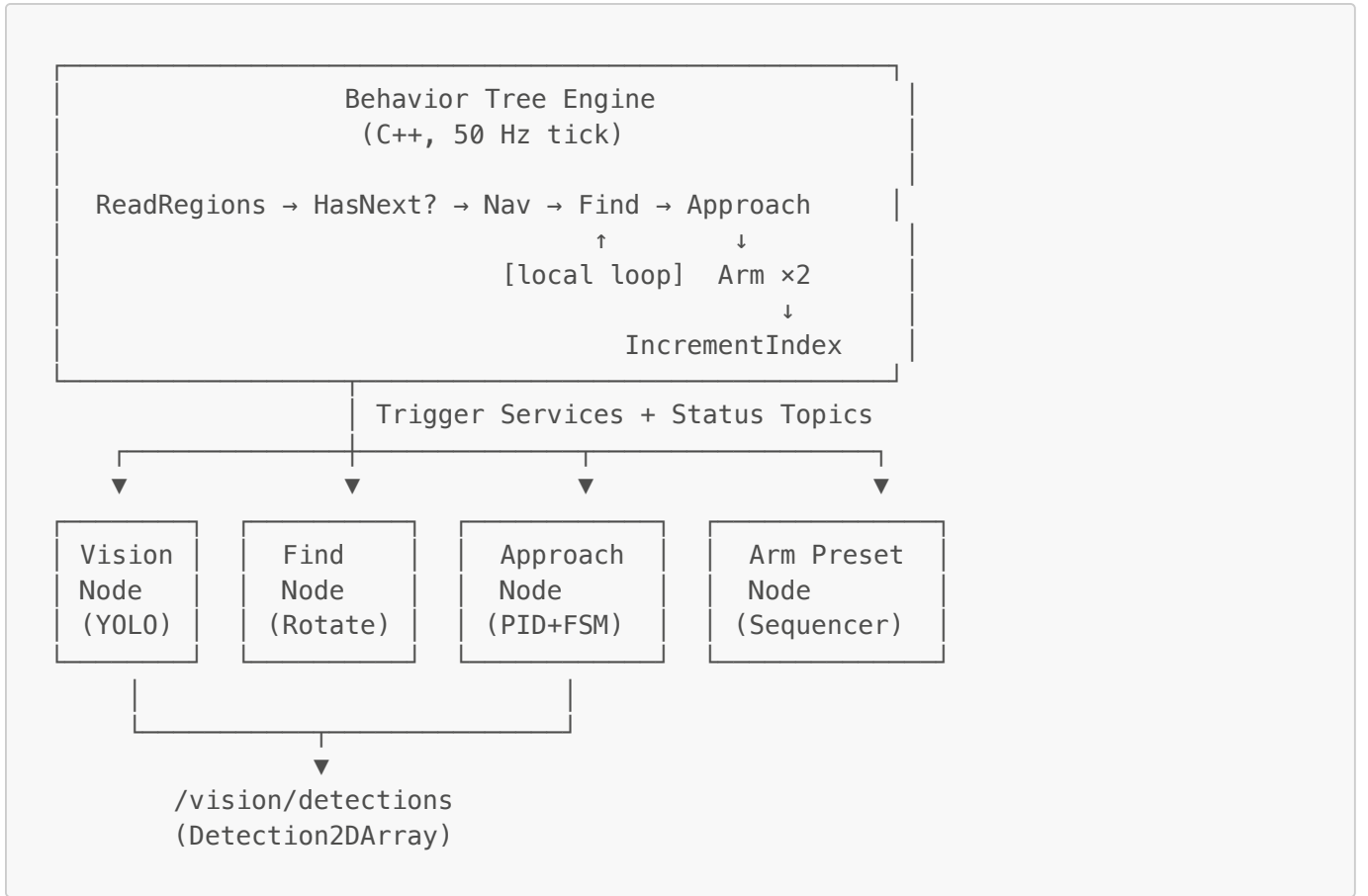
8.2 Deliberative-Reactive Hybrid

The Behavior Tree provides the deliberative layer: "go to region A, then search, then approach, then grab." The Python nodes provide the reactive layer: "rotate while searching," "track the target while approaching," "stop if lost." This avoids the brittleness of pure deliberative planning while retaining the predictability of a structured mission.

8.3 Retry-Until-Exhausted Pattern

The Behavior Tree wraps the global find-approach-grab sequence in `KeepRunningUntilFailure`. When the global sequence succeeds and the local pickup loop eventually fails (no more visible objects), the tree naturally proceeds to the next region. This elegant idiom enables opportunistic local pickups without explicit counting or state tracking.

8.4 System Graph



9. Map Partitioning Pipeline

The offline map partitioning pipeline (`map_partitioner.py`) transforms an occupancy grid map into an ordered sequence of reachable region centers. It proceeds in five stages.

9.1 Stage 1: Preprocessing

Input: Occupancy grid map (PGM + YAML, ROS convention)

Step 1.1 — Obstacle inflation: Occupied cells (value = 1) are dilated by the robot's radius (`inflation_radius` in pixels) using a morphological elliptical kernel:

$$O_{\text{inflated}} = O \oplus K_r$$

where $\oplus$ is morphological dilation and K_r is an elliptical structuring element with radius r .

Step 1.2 — Connected component extraction: Free space is defined as the complement of inflated obstacles. The connected component containing the home pixel is extracted:

$$F_{\text{reachable}} = \{p \in F_{\text{free}} \mid \text{connected}(p, \text{home})\}$$

This ensures the robot is only assigned regions it can physically reach.

9.2 Stage 2: Seed Detection

The Euclidean distance transform is computed on the reachable free space:

$$D(p) = \min_{q \in O_{\text{inflated}}} \|p - q\|_2$$

Local maxima of $D(p)$ — pixels whose distance value is greater than all neighbors within a minimum separation distance — are selected as region seeds:

$$S = \{p \in F_{\text{reachable}} \mid D(p) \geq D(q) \forall q \in N_d(p)\}$$

where $N_d(p)$ is the neighborhood of p within physical distance d (default 0.3 m). These peaks correspond to the "interior" points of rooms and corridors — locally maximally distant from obstacles.

Physical intuition: The distance transform creates a "mountain range" where ridges correspond to the medial axis of free space. Local maxima are the "peaks" of this range — the most interior points of each room-like subspace.

9.3 Stage 3: Voronoi Partition

Marker-based watershed segmentation is applied to the inverted distance transform:

$$L(p) = \text{Watershed}(-D(p), \text{markers} = S, \text{mask} = F_{\text{reachable}})$$

The inverted distance transform $-D(p)$ creates a topographic surface where each seed is a local minimum. Water flowing from each pixel flows downhill to the nearest seed, producing a Voronoi-like tessellation that respects obstacle boundaries.

9.4 Stage 4: Greedy Merge

Small regions (below `min_area`) are forcibly merged into their largest neighbor. Then, an iterative greedy merge reduces the partition to a manageable number of regions.

Merge cost function:

$$C(r_1, r_2) = \alpha \left(\frac{1}{A_1} + \frac{1}{A_2} \right) + \beta (d_{\text{path}}(\text{home}, r_1) + d_{\text{path}}(\text{home}, r_2))$$

where:

- A_i is the area (pixel count) of region i
- d_{path} is the A* path length from home to the region's center
- α penalizes small regions (encouraging their merger)
- β penalizes regions far from home

A **hard radius constraint** enforces a maximum region radius (default 1.0 m). Any merge whose estimated combined radius would exceed this limit is rejected (cost = ∞). This ensures the robot's camera can cover an entire region from its center without missing corners.

The algorithm iteratively selects the edge with minimum merge cost below the threshold, merges the two regions, and recomputes affected costs.

9.5 Stage 5: TSP Routing

Given n region centers $C = c_1, \dots, c_n$, a pairwise distance matrix is computed using A* pathfinding on the free-space mask:

$$D_{ij} = |A^*(c_i, c_j)|$$

A nearest-neighbor greedy TSP tour starting from home determines the visitation order:

```
unvisited = {1, ..., n}
current = home
```

```

while unvisited:
    next = argmin_{i ∈ unvisited} distance(current, c_i)
    append next to tour
    unvisited.remove(next)
    current = c_next

```

This is a $\frac{1}{2}(\lceil \log_2 n \rceil + 1)$ -approximation to the optimal metric TSP tour, though for practical region counts ($n \leq 20$) the greedy heuristic is sufficient.

9.6 A* Pathfinding

All path queries use A* on an 8-connected grid with Euclidean distance heuristic:

$$f(p) = g(p) + h(p)$$

where $g(p)$ is the actual path cost from start (1.0 for cardinal moves, $\sqrt{2} \approx 1.414$ for diagonal moves) and $h(p)$ is the Euclidean distance to goal. This is admissible and consistent, guaranteeing optimal paths.

Reachability back-off: When a region's geometric centroid is unreachable (e.g., separated by an inflated obstacle that was not visible before inflation), the algorithm back-tracks along the ray from the home pixel toward the centroid, sampling pixels within the same region until a reachable one is found.

10. Vision and Detection

10.1 YOLO Inference Pipeline

The vision node subscribes to the camera image topic and runs YOLO inference each frame:

1. **Image acquisition:** `sensor_msgs/Image` → OpenCV `bgr8` via `cv_bridge`
2. **Inference:** `model.predict(source, conf, iou, imgsz)` with `verbose=False` for performance
3. **Message construction:** Bounding boxes (xywh format) are packed into `vision_msgs/Detection2DArray` with class ID and confidence score
4. **Debug output:** Annotated frames are published to `vision/annotated_image` for visualization in `rqt_image_view` or `RViz`

10.2 Coordinate Convention

The vision pipeline uses the standard ROS `vision_msgs` convention:

- `bbox.center.position.x`, `bbox.center.position.y` — pixel coordinates of the bounding box center
- `bbox.size_x`, `bbox.size_y` — width and height of the bounding box in pixels
- Origin is at the top-left corner of the image

The approach controller interprets `x` as horizontal error (for yaw correction) and `y` as depth proxy (larger `y` = closer to bottom of image = closer to robot).

11. Object Search with Multi-Frame Debounce

11.1 Problem Statement

When searching for objects by rotation, a single-frame YOLO detection can be a false positive. Acting on a spurious detection wastes time and degrades reliability.

11.2 Debounce Algorithm

The `find_node` implements a **consecutive-frame confirmation** (debounce) filter:

```
confirm_count = 0
on each detection frame:
    if any detection.confidence >= min_confidence:
        confirm_count += 1
        if confirm_count >= required_confirm_frames:
            report SUCCESS
    else:
        confirm_count = 0 // reset on any negative frame
```

State machine:

```
IDLE —[start service called]—> SEARCHING
SEARCHING —[confirm_count >= N]—> SUCCESS
```

During SEARCHING, the robot publishes a constant angular velocity `Twist.angular.z = rotate_speed`. Upon SUCCESS, a zero Twist is published (emergency stop).

11.3 Parameter Selection

- `required_confirm_frames` (default 4): With a 10 Hz timer, this requires ~400 ms of continuous detection. Higher values increase robustness against false positives but increase search time.
- `min_confidence` (default 0.6): Should be higher than the vision node's `conf_threshold` (0.5). The vision node publishes all plausible candidates; the find node only confirms high-confidence ones.

11.4 False Positive Probability

Assuming independent frame errors with false-positive probability p per frame:

$$P(\text{false confirm}) = p^N$$

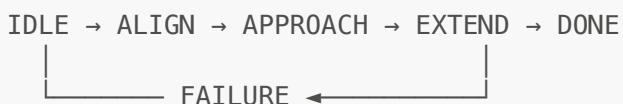
For $p = 0.1$ and $N = 4$: $P \approx 1 \times 10^{-4}$, a 10,000× reduction from single-frame detection.

12. Approach Controller with Blind-Zone Takeover

12.1 Problem Statement

The RoboMaster EP camera is mounted above the gripper. As the robot approaches an object, the object eventually drops below the camera's field of view — entering a "blind zone" where visual tracking is impossible. The approach controller must handle this transition smoothly.

12.2 State Machine



IDLE: Wait for a stable detection lock. The node selects the **lowest** object in the frame (largest y-coordinate, i.e., closest to the robot) and requires `required_confirm_frames` (default 3) consecutive detections before locking.

ALIGN: Apply proportional yaw control to center the target horizontally:

$$\omega = K_p \cdot (x_{\text{target}} - x_{\text{current}})$$

where K_p is the yaw proportional gain (`yaw_kp`, default 0.002) and x_{target} is the desired horizontal pixel position (calibrated to the gripper center). Transition to APPROACH when $|x_{\text{error}}| < \text{align_threshold}$ (15 pixels).

APPROACH: Drive forward with simultaneous yaw correction:

$$v = v_{\text{forward}}, \quad \omega = K_p \cdot (x_{\text{target}} - x_{\text{current}})$$

Transition to EXTEND when `target_y > stop_y_threshold` (the object enters the blind zone, typically ~280 pixels from top). The y-coordinate threshold acts as a proxy for physical proximity.

EXTEND: Continue driving forward blindly for a calibrated duration:

$$t_{\text{extend}} = \frac{d_{\text{extend}}}{v_{\text{forward}}}$$

This compensates for the remaining distance the robot must travel after losing visual contact. The extend distance (default 0.30 m) should approximately equal the blind-zone depth.

DONE: Stop and report SUCCESS.

12.3 Target Tracking with EMA Filter

To suppress detection jitter, the locked target position is filtered with an exponential moving average (EMA):

$$x_{\text{locked}}^{(t)} = \alpha \cdot x_{\text{detected}} + (1 - \alpha) \cdot x_{\text{locked}}^{(t-1)}$$

with smoothing factor $\alpha = 0.6$ (higher = more responsive, lower = smoother).

12.4 Target Association

During ALIGN and APPROACH states, detections are associated with the locked target via **nearest-neighbor matching within a tracking radius** (`tracking_threshold`, default 120 pixels):

$$\text{match} = \arg \min_i \sqrt{(x_i - x_{\text{locked}})^2 + (y_i - y_{\text{locked}})^2}$$

Detections outside the radius are ignored. If no match is found within the radius for `timeout_lost` seconds (default 1.0 s), the node reports FAILURE.

12.5 Smart Blind-Zone Transition

When tracking is lost during APPROACH, the controller applies a heuristic: if the last known target y-position was close to the blind-zone threshold (within 60 pixels), the target likely entered the blind zone rather than being genuinely lost. In this case, the state machine transitions to EXTEND rather than reporting FAILURE:

```
if state == APPROACH and time_since_last_detection > timeout_lost:
    if last_target_y > stop_y_threshold - 60:
        transition to EXTEND // blind zone takeover
    else:
        report FAILURE // genuinely lost at distance
```

12.6 PID Gain Tuning

The yaw proportional controller uses a single gain $K_p = 0.002$. For a 640-pixel image, a 100-pixel error produces $\omega = 0.2 \text{ rad/s}$ (~11.5 deg/s). This is intentionally conservative to avoid overshoot during approach.

The full transfer function from pixel error to angular velocity is:

$$G(s) = K_p = 0.002 \text{ (rad/s)/pixel}$$

13. Behavior Tree Orchestration

13.1 Why Behavior Trees?

Behavior Trees (BTs) offer several advantages over finite state machines (FSMs) for robot mission control:

- 1. **Modularity:** Each node encapsulates a single behavior and can be reused across trees.
- 2. **Reactivity:** BTs re-evaluate conditions on every tick (50 Hz), enabling responsive recovery from failures.
- 3. **Composability:** Complex behaviors emerge from simple building blocks (Sequence, Fallback, Decorators).
- 4. **Readability:** The XML representation is human-readable and modifiable without recompilation.

13.2 Tree Structure Semantics

Control Flow Nodes

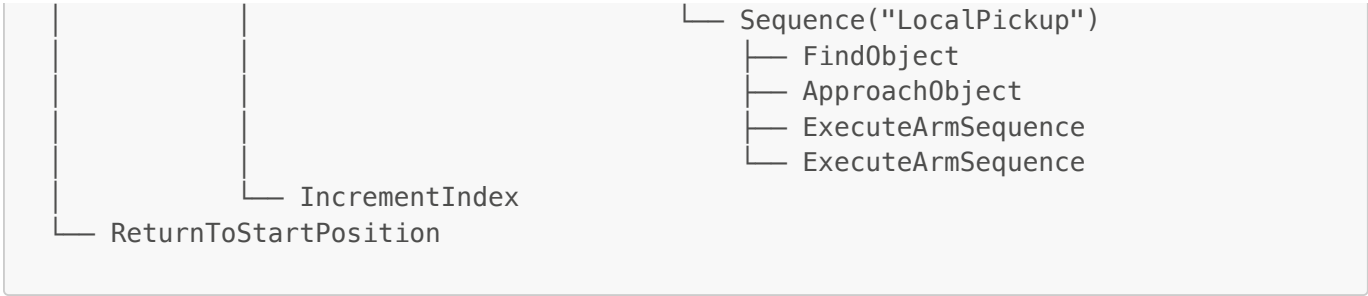
Node	Children	Returns SUCCESS when	Returns FAILURE when
Sequence	N	All children succeed	Any child fails
ForceSuccess	1	Always SUCCESS (regardless of child)	Never
KeepRunningUntilFailure	1	Never (loops)	Child returns FAILURE

Custom Action Nodes

See Section 8.1 for the complete list of custom nodes and their ROS interactions.

13.3 Main Tree: Complete Mission Logic

```
Sequence("Root")
├── ReadRegionCenters(regions, current_index)    // Load region list
├── ForceSuccess
├── └── KeepRunningUntilFailure("ProcessAllCenters")
│       ├── Sequence("ProcessOneCenter")
│       │   ├── HasNextCenterPoint?            // Guard: more regions?
│       │   ├── ForceSuccess
│       │   └── └── KeepRunningUntilFailure("CenterConfirmationLoop")
│       │           ├── Sequence("GlobalCheck")
│       │           │   ├── NavToIndexedPoseClient // Navigate to region
│       │           │   ├── FindObject             // Search for object
│       │           │   ├── ForceSuccess
│       │           │   └── └── Sequence("GlobalGrabAndLocalClean")
│       │           │           ├── ApproachObject // Approach target
│       │           │           ├── ExecuteArmSequence // Grab
│       │           │           ├── ExecuteArmSequence // Retract
│       │           │           └── ForceSuccess
│       │           └── └──
│       └── └── KeepRunningUntilFailure("LocalPickupLoop")
```



13.4 Key Design Patterns

Pattern 1: Retry-Until-Exhausted

```
<ForceSuccess>
  <KeepRunningUntilFailure>
    <Sequence>
      <FindObject/>
      <ApproachObject/>
      <ExecuteArmSequence/>
    </Sequence>
  </KeepRunningUntilFailure>
</ForceSuccess>
```

KeepRunningUntilFailure re-ticks its child **Sequence** as long as it succeeds. When **FindObject** eventually fails (no more objects visible), the decorator returns **FAILURE**. **ForceSuccess** converts this to **SUCCESS**, allowing the tree to proceed to the next region. This cleanly encodes "keep picking until nothing is left."

Pattern 2: Global-then-Local

The tree first executes a **global** find-approach-grab at the region center, then enters a **local** loop that searches for remaining objects without re-navigating. This minimizes unnecessary navigation while maximizing pickup yield per region.

Pattern 3: Timeout-Bounded Actions

Each custom action accepts a **timeout** input port. The C++ action node times out and returns **FAILURE** if the Python node hasn't reported **SUCCESS** within the timeout period. This prevents the tree from hanging indefinitely.

13.5 Blackboard Variables

The Behavior Tree uses a shared blackboard for inter-node communication:

Variable	Type	Producer	Consumer
regions	vector<pair<double,double>>	ReadRegionCenters	HasNextCenterPoint, NavToIndexedPoseClient
current_index	int	ReadRegionCenters, IncrementIndex	HasNextCenterPoint, NavToIndexedPoseClient

14. Inter-Process Communication Protocol

14.1 Protocol Definition

Every Python behavior node implements a uniform IPC protocol:

Service: `~/start` (`std_srvs/Trigger`)

- Request: empty
- Response: `success=true/false, message="descriptive string"`
- Semantics: Start the behavior. Return immediately; execution is asynchronous.

Topic: `~/status` (`std_msgs/String`)

- Published values: `"SEARCHING", "ALIGNING", "APPROACHING", "WAITING_FOR_DETECTION", "EXECUTING", "SUCCESS", "FAILURE", "FAILURE_NO_SERVER"`
- Semantics: Current execution state; `"SUCCESS"` and any string containing `"FAILURE"` are terminal states.

14.2 C++ Client Pattern

Each C++ BT action node follows the same pattern:

```
onStart():
    send async Trigger request to ~/start
    if response.success: return RUNNING
    else: return FAILURE

onRunning():
    spin_some(node) // process incoming status messages
    if latest_status == "SUCCESS": return SUCCESS
    if "FAILURE" in latest_status: return FAILURE
    if elapsed > timeout: return FAILURE // timeout guard
    return RUNNING // keep polling
```

14.3 Rationale for Asynchronous Design

Long-running behaviors (search rotation, approach driving, arm sequencing) cannot block the ROS event loop. The Trigger + Status pattern solves this by:

- Starting the behavior synchronously (2-second timeout for service call)
- Polling asynchronously on each BT tick (50 Hz)
- The Python node runs its own timer/control loop independently

15. Arm Preset Sequencing

15.1 Design

The arm controller executes a configurable sequence of preset positions. Each preset is a 2D point in the arm's Cartesian workspace (x = forward extension, z = vertical lift).

15.2 Concurrency Model

The arm node uses a **multi-threaded executor with reentrant callback groups**:

- **ReentrantCallbackGroup**: Allows multiple service calls to be processed concurrently. The `MoveArm` action client calls block the calling thread but don't block the executor.
- **MultiThreadedExecutor**: Runs multiple executor threads, allowing status publishing and service handling to proceed while an action goal is pending.

- **Threading.Lock:** Protects the **active** flag to prevent overlapping sequences.

15.3 Execution Flow

```

handle_start():
    if already active: reject
    set active = true
    spawn background thread

background thread:
    wait for MoveArm action server
    for each preset_name in sequence:
        send MoveArm goal
        block until result
        if failed and emergency_stop_on_error:
            send home goal
            return FAILURE
    return SUCCESS

```

15.4 Kinematic Limits

Each preset is validated at startup against configurable joint limits. Out-of-range presets generate a warning but do not prevent execution.

Appendix A: Parameter Tuning Guide

A.1 Improving Detection Reliability

If the robot frequently fails to find objects:

1. **Lower **min_confidence**** in `find_node` (e.g., from 0.6 to 0.5)
2. **Decrease **required_confirm_frames**** (e.g., from 4 to 3)
3. **Verify YOLO model quality** by running `vision_node` standalone and viewing **vision/annotated_image** in `rqt_image_view`

A.2 Improving Approach Success Rate

If the approach frequently fails:

1. **Calibrate **target_x_offset****: Place an object in front of the robot. In the annotated image, note the x-coordinate of the object's center when the gripper is aligned with it. Set **target_x_offset** to this value.
2. **Tune **stop_y_threshold****: Observe at what y-coordinate the object disappears from the camera during approach. Set **stop_y_threshold** slightly above this value.
3. **Adjust **extend_dist****: Measure the physical distance the robot travels between losing visual contact and being within grasping range. Set this as **extend_dist**.
4. **Increase **timeout_lost**** if the target is frequently occluded during approach (e.g., from 1.0 to 1.5).

A.3 Improving Navigation Reliability

1. **Increase **inflation_radius**** in `map_partitioner.py` if the robot consistently gets too close to obstacles.
2. **Decrease **max_region_radius**** if regions are too large for the camera's field of view.

3. **Adjust `alpha` / `beta`** in merge cost: higher alpha favors merging small regions; higher beta penalizes regions far from home.

A.4 Tuning the Arm Sequence

1. **Adjust preset positions** by physically moving the arm to desired positions and recording the x/z values from the RoboMaster SDK.
2. **Modify `default_sequence`** to change the grasp motion: e.g., `["home", "forward", "down", "backward", "home"]` for a scooping motion.

Appendix B: File Reference

File	Role
<code>brickpick/vision_node.py</code>	YOLO inference; publishes <code>Detection2DArray</code> and debug images
<code>brickpick/find_node.py</code>	In-place rotation search with multi-frame debounce
<code>brickpick/approach_node.py</code>	5-state approach controller with blind-zone takeover
<code>brickpick/arm_preset_node.py</code>	Preset-sequence arm controller with reentrant callbacks
<code>brickpick/map_partitioner.py</code>	Offline Voronoi + A* + TSP map partitioning
<code>brickpick/odom_2d_filter.py</code>	6-DoF to 2D odometry projection
<code>brickpick/teleop_keyboard_node.py</code>	Incremental keyboard teleoperation
<code>brickpick/image_capture_node.py</code>	Keyboard-triggered dataset collection
<code>src/bt_executor.cpp</code>	Behavior Tree engine main loop
<code>src/bt_nodes_plugin.cpp</code>	Custom BT node registration
<code>src/find_action.cpp</code>	BT action: trigger <code>find_node</code> , poll status
<code>src/approach_action.cpp</code>	BT action: trigger <code>approach_node</code> , poll status
<code>src/arm_action.cpp</code>	BT action: trigger <code>arm_preset_node</code> , poll status
<code>src/nav_to_indexed_pose_client.cpp</code>	BT action: Nav2 <code>NavigateToPose</code> for indexed region
<code>src/reading_region.cpp</code>	BT action: parse <code>region_centers.txt</code> to blackboard
<code>src/has_next_center_point.cpp</code>	BT condition: check index bounds
<code>src/increment_index.cpp</code>	BT action: increment blackboard index
<code>src/return_to_start_position.cpp</code>	BT action: navigate to home pose
<code>config/brickpick_tree.xml</code>	Full multi-region Behavior Tree with local loops
<code>config/simple_pick_tree.xml</code>	Minimal single-cycle Behavior Tree for debugging
<code>config/vision_params.yaml</code>	Vision node configuration
<code>config/approach_params.yaml</code>	Approach node configuration
<code>config/arm_presets.yaml</code>	Arm preset positions and limits

File	Role
config/nav2_params.yaml	Nav2 planner/controller configuration
config/mapper_params_online_async.yaml	slam_toolbox online SLAM parameters
launch/main.launch.py	Main launch file for full pipeline